



STM32F4 四轴飞行器开发手册

原理与算法部分

摘要

本文档对四轴飞行器的控制原理与算法部分进行了详细讲解，包括常用的姿态算法解析，能对开发者形成较大帮助。

光电 209 实验室

目 录

1. 图示.....	3
2. 数学模型	4
2.1 四旋翼无人机模型	4
2.2 状态空间表达式	7
3. LQR 控制器	9
3.1 概述	9
3.2 实例	11
4. PID 控制器	18
4.1 控制方案与原理	18
4.2 PID 参数调试方法介绍	19
5. 附录	21
5.1 升力与 PWM 值的关系	21
5.2 四元数的求导公式	22
5.3 四元数与欧拉角的关系	23
5.4 显式互补滤波 (ECF)	24
5.5 前馈补偿器	25
5.6 高通滤波器	25
5.7 雷达定位算法	27
5.8 光流数据处理	32
5.9 传统互补滤波 (CF)	34

1. 图示



图 1-1 四旋翼无人机的实物图

2. 数学模型

2.1 四旋翼无人机模型

为了得到系统的状态空间表达式，首先需要对该系统进行动力学建模。建立如图 2-1 所示的直角坐标系，规定相关符号如表 2-1 所示。

表 2-1 符号说明

m	整机的质量(kg)
φ	绕 X_b 轴旋转的角度，即横滚角(rad)
θ	绕 Y_b 轴旋转的角度，即俯仰角(rad)
$\vec{\omega}_B$	刚体坐标系下的角速度(rad/s)
ψ	绕 Z_b 轴旋转的角度，即偏航角(rad)
f_i	第 i 个桨叶产生的升力(N)
C_i	第 i 个桨叶产生的转矩(N·m)
b	机架的半轴距(m)
I	机体的惯性矩阵(kg·m ²)
C_t	单桨综合升力系数(N/(rad/s) ²)
C_m	单桨综合力矩系数(N·m/(rad/s) ²)

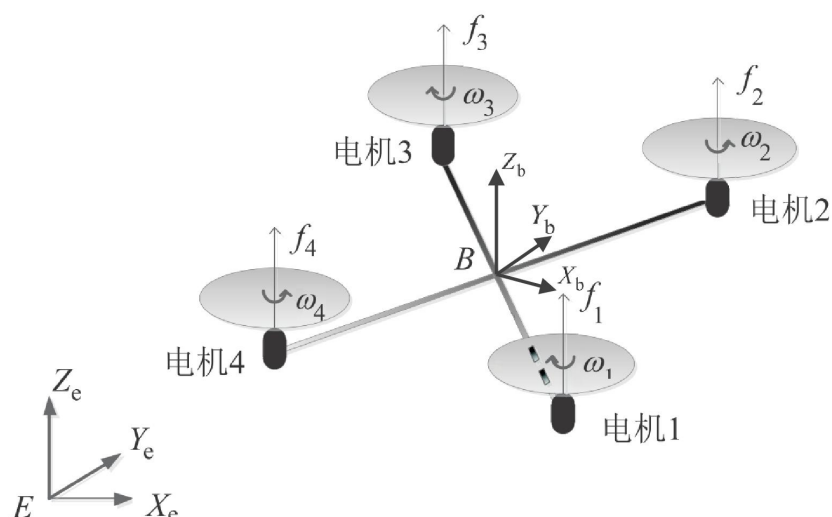


图 2-1 坐标系的建立

以 $E = \{X_e \ Y_e \ Z_e\}$ 表示世界坐标系， $B = \{X_b \ Y_b \ Z_b\}$ 表示机体坐标系，

定义世界坐标系的基向量为 $\vec{e}_{x_e}, \vec{e}_{y_e}, \vec{e}_{z_e}$ ，刚体坐标系的基向量为 $\vec{e}_{x_b}, \vec{e}_{y_b}, \vec{e}_{z_b}$ ，欧拉角 $\Phi = \{\varphi \ \theta \ \psi\}$ 。采用 $\mathbf{Z}-\mathbf{Y}-\mathbf{X}$ 欧拉角表示法，则有从世界坐标系到机体坐标系的旋转矩阵：

$$R = \begin{bmatrix} \cos \psi \cos \theta & \cos \psi \sin \theta \sin \varphi - \sin \psi \cos \varphi & \cos \psi \sin \theta \cos \varphi + \sin \psi \sin \varphi \\ \sin \psi \cos \theta & \sin \psi \sin \theta \sin \varphi + \cos \psi \cos \varphi & \sin \psi \sin \theta \cos \varphi - \cos \psi \sin \varphi \\ -\sin \theta & \sin \varphi \cos \theta & \cos \varphi \cos \theta \end{bmatrix}$$

如图 2-1 所示，四旋翼无人机的 1、3 旋翼为逆时针旋转，而 2、4 旋翼为顺时针旋转，因此当 4 个电机转速一致且升力之和等于机体自重时，无人机可以保持悬停，当升力同时增加或减少时，无人机可上升或下降。保持 4 个旋翼升力和与无人机自重相等，1、3 旋翼增速（减速）且 2、4 旋翼减速（增速），由于转矩差的作用，无人机可实现偏航运动；1、2 旋翼增速（减速）且 3、4 旋翼减速（增速），无人机可实现俯仰运动；1、4 旋翼增速（减速）且 2、3 旋翼减速（增速），无人机可实现横滚运动。将以上的基本动作进行组合，无人机可实现各种复杂运动。

牛顿定律和欧拉方程为我们提供了 6 自由度刚体的移动和转动动力学方程

$$\begin{cases} \vec{F} = m \cdot \vec{a} \\ \vec{M}_B = \mathbf{I} \cdot \dot{\vec{\omega}}_B + \vec{\omega}_B \times (\mathbf{I} \cdot \vec{\omega}_B) \end{cases} \quad (1)$$

1) 合外力 $\vec{F}$ 由重力和 4 个旋翼产生的推力叠加而成：

$$\begin{aligned} \vec{F} &= [\vec{e}_{x_e} \ \vec{e}_{y_e} \ \vec{e}_{z_e}] \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + [\vec{e}_{x_b} \ \vec{e}_{y_b} \ \vec{e}_{z_b}] \begin{bmatrix} 0 \\ 0 \\ F_t \end{bmatrix} \\ &= [\vec{e}_{x_e} \ \vec{e}_{y_e} \ \vec{e}_{z_e}] \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + [\vec{e}_{x_e} \ \vec{e}_{y_e} \ \vec{e}_{z_e}] R \begin{bmatrix} 0 \\ 0 \\ F_t \end{bmatrix} \\ &= [\vec{e}_{x_e} \ \vec{e}_{y_e} \ \vec{e}_{z_e}] \begin{bmatrix} F_t(\cos \psi \sin \theta \cos \varphi + \sin \psi \sin \varphi) \\ F_t(\sin \psi \sin \theta \cos \varphi - \cos \psi \sin \varphi) \\ F_t \cos \varphi \cos \theta - mg \end{bmatrix} \end{aligned}$$

则

$$\begin{cases} \ddot{x} = \frac{F_t}{m}(\cos \psi \sin \theta \cos \varphi + \sin \psi \sin \varphi) \\ \ddot{y} = \frac{F_t}{m}(\sin \psi \sin \theta \cos \varphi - \cos \psi \sin \varphi) \\ \ddot{z} = \frac{F_t}{m} \cos \varphi \cos \theta - g \end{cases} \quad (2)$$

其中, $F_t = f_1 + f_2 + f_3 + f_4$ 。

2) 合力矩 $\vec{M}_B$ 由推力力矩和 4 个电机给机身的反扭矩叠加而成:

$$M_B = \begin{bmatrix} I_{x_b} & 0 & 0 \\ 0 & I_{y_b} & 0 \\ 0 & 0 & I_{z_b} \end{bmatrix} \cdot \begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} + \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times \left(\begin{bmatrix} I_{x_b} & 0 & 0 \\ 0 & I_{y_b} & 0 \\ 0 & 0 & I_{z_b} \end{bmatrix} \cdot \begin{bmatrix} p \\ q \\ r \end{bmatrix} \right) = \begin{bmatrix} I_{x_b} \dot{p} + (I_{z_b} - I_{y_b})qr \\ I_{y_b} \dot{q} + (I_{x_b} - I_{z_b})pr \\ I_{z_b} \dot{r} + (I_{y_b} - I_{x_b})pq \end{bmatrix}$$

其中

$$M_B = \begin{bmatrix} \frac{\sqrt{2}}{2}b(-f_1 + f_2 + f_3 - f_4) \\ \frac{\sqrt{2}}{2}b(-f_1 - f_2 + f_3 + f_4) \\ C_1 - C_2 + C_3 - C_4 \end{bmatrix}$$

解得

$$\begin{cases} \dot{p} = \frac{\frac{\sqrt{2}}{2}b(-f_1 + f_2 + f_3 - f_4)}{I_{x_b}} + \frac{I_{y_b} - I_{z_b}}{I_{x_b}}qr \\ \dot{q} = \frac{\frac{\sqrt{2}}{2}b(-f_1 - f_2 + f_3 + f_4)}{I_{y_b}} + \frac{I_{z_b} - I_{x_b}}{I_{y_b}}pr \\ \dot{r} = \frac{C_1 - C_2 + C_3 - C_4}{I_{z_b}} + \frac{I_{x_b} - I_{y_b}}{I_{z_b}}pq \end{cases} \quad (3)$$

又因为欧拉角角速度与刚体坐标系下的角速度有如下的关系:

$$\begin{bmatrix} \dot{\varphi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin \varphi \tan \theta & \cos \varphi \tan \theta \\ 0 & \cos \varphi & -\sin \varphi \\ 0 & \frac{\sin \varphi}{\cos \theta} & \frac{\cos \varphi}{\cos \theta} \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (4)$$

当无人机处于平衡状态的时候, 可以认为

$$\begin{cases} \varphi, \theta \rightarrow 0 \\ p, q, r \rightarrow 0 \\ F_t \rightarrow mg \end{cases} \quad (5)$$

则式(4)变为

$$\begin{bmatrix} \dot{\varphi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \quad (6)$$

联立式(2)(3)(5)(6), 则可得线性化后的模型:

$$\left\{ \begin{array}{l} \ddot{x} = g\theta \\ \ddot{y} = -g\varphi \\ \ddot{z} = \frac{u_1}{m} \\ \ddot{\varphi} = \frac{u_2}{I_{x_b}} \\ \ddot{\theta} = \frac{u_3}{I_{y_b}} \\ \ddot{\psi} = \frac{u_4}{I_{z_b}} \\ u_1 = f_1 + f_2 + f_3 + f_4 - mg \\ u_2 = \frac{\sqrt{2}}{2}b(-f_1 + f_2 + f_3 - f_4) \\ u_3 = \frac{\sqrt{2}}{2}b(-f_1 - f_2 + f_3 + f_4) \\ u_4 = C_1 - C_2 + C_3 - C_4 \end{array} \right. \quad (7)$$

2.2 状态空间表达式

式(7)实际上可以看成是 6 个子系统，有 3 个关于位置的子系统，有 3 个关于姿态的子系统，则有以下 6 个状态空间表达式：

a) X 轴方向移动子系统：

$$\left\{ \begin{array}{l} \begin{bmatrix} \dot{x} \\ \ddot{x} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix} + \begin{bmatrix} 0 \\ g \end{bmatrix} \theta \\ y_x = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \end{bmatrix} \end{array} \right. \quad (8)$$

b) Y 轴方向移动子系统：

$$\left\{ \begin{array}{l} \begin{bmatrix} \dot{y} \\ \ddot{y} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} y \\ \dot{y} \end{bmatrix} + \begin{bmatrix} 0 \\ -g \end{bmatrix} \varphi \\ y_y = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} y \\ \dot{y} \end{bmatrix} \end{array} \right. \quad (9)$$

c) Z 轴方向移动子系统：

$$\left\{ \begin{array}{l} \begin{bmatrix} \dot{z} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} z \\ \dot{z} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u_1 \\ y_z = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} z \\ \dot{z} \end{bmatrix} \end{array} \right. \quad (10)$$

d) 横滚运动子系统：

$$\begin{cases} \begin{bmatrix} \dot{\varphi} \\ \ddot{\varphi} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \varphi \\ \dot{\varphi} \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ I_{x_b} \end{bmatrix} u_2 \\ y_{\varphi} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \varphi \\ \dot{\varphi} \end{bmatrix} \end{cases} \quad (11)$$

e) 俯仰运动子系统:

$$\begin{cases} \begin{bmatrix} \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ I_{y_b} \end{bmatrix} u_3 \\ y_{\theta} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix} \end{cases} \quad (12)$$

f) 偏航运动子系统:

$$\begin{cases} \begin{bmatrix} \dot{\psi} \\ \ddot{\psi} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \psi \\ \dot{\psi} \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ I_{z_b} \end{bmatrix} u_4 \\ y_{\psi} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \psi \\ \dot{\psi} \end{bmatrix} \end{cases} \quad (13)$$

3. LQR 控制器

3.1 概述

LQR(Linear Quadratic Regulator), 即线性二次型调节器。LQR 可得到状态线性反馈的最优控制规律, 易于构成闭环最优控制。LQR 最优控制利用廉价成本可以使原系统达到较好的性能指标且易于实现, 是现代控制理论中发展较为成熟的一部分。下面对其原理做一个简单的概述后将其应用于四旋翼无人机的姿态控制过程中。

设连续线性时不变系统 (假设系统完全能控) 的状态空间表达式为

$$\begin{cases} \dot{x} = Ax + Bu \\ y = Cx + Du \end{cases} \quad (14)$$

在现代控制理论中, 最基本的控制系统就是全状态反馈控制系统 (系统框图如图 3-1 所示), 其目标是设计一个状态反馈控制器 $u = -Kx$ 来控制系统的表现。

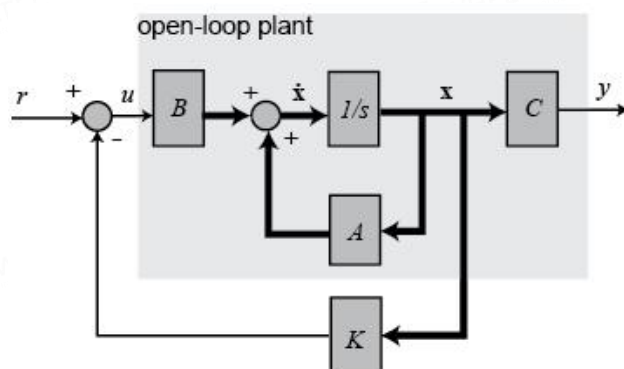


图 3-1 全状态反馈控制系统

现在的问题是这样的 K 不是唯一的, 那么, 怎么样的 K 才是最优的呢? 在引入代价函数之前, 先来看这样的问题 (来自于 MATLAB 官方教程): 假设你在寻找从家里到公司的最佳方式, 并且有多种交通方式可以选择, 你可以开车、骑自行车、坐公交车以及租直升飞机。那么, 哪种交通方式是你最佳的选择? 这个问题本身没有答案, 因为我们不知道什么是“最佳”。假设现在只考虑时间成本和金钱成本, 将 4 种交通方式的信息填入如下的表格。

表 3-1

Transportation	Time / min	Money / \$
Car	20	45

Bike	75	0
Bus	30	12
Helicopter	4	2578

可以看到，如果时间成本是我们最关注的，那么显然租直升飞机是最佳的选择。反之，如果我们更关注金钱成本，那么骑自行车将是最佳的选择。当然，在现实生活中，人们往往会追求两者的平衡。比如：今天你有个早会，你理所当然会关注时间成本，但另一方面，你也没有足够富裕，因此也要关注金钱成本。这样的话，如何进行选择呢？这里有个简单的量化评估方法，可以建立这样一个评价函数 $J = Q \cdot Time + R \cdot Money$ ，根据个人的需要设置不同的权重。如果权重设置如表 3-2 所示，那么，最优选择为开车去上班。如果权重设置如表 3-3 所示，那么，最优选择为坐公交车去上班。

表 3-2

Transportation	Q	Time / min	R	Money / \$	J(Cost)
Car	10	20	1	45	245
Bike	10	75	1	0	750
Bus	10	30	1	12	312
Helicopter	10	4	1	2578	2618

表 3-3

Transportation	Q	Time / min	R	Money / \$	J(Cost)
Car	5	20	5	45	325
Bike	5	75	5	0	375
Bus	5	30	5	12	210
Helicopter	5	4	5	2578	12910

类似的，对于控制系统，我们引入代价函数(Quadratic Cost Function)

$$J = \int_0^{+\infty} [x^T(t)Qx(t) + u^T(t)Ru(t)] dt \quad (15)$$

式中， Q 为半正定的状态加权矩阵， R 为正定的控制加权矩阵。在工程实际中， Q 和 R 是对称矩阵且常取对角阵。 Q 中对角线上的元素 q_i 表示对相应误差分量 x_i （最终的目标是让每一个状态变量的值都变为 0，所以每一个状态变量的值又称为误差值）的重视程度，越被重视的误差分量，希望它越快的变小，相应地，其权重系数就取得越大。类似的， R 中对角线上的元素 r_i 表示对相应输入分量 u_i 的限制，让其不要太大。LQR 控制器的设计思路就是设计一个状态反馈控制器

$u = -Kx$, 使得代价函数最小 $\min J$, 从而达到利用廉价成本使原系统达到较好的性能指标的目的。至于状态反馈矩阵 K 怎么计算, 这里只给出相应结论:

$$K = R^{-1}B^T P \quad (16)$$

其中, P 是 **Riccati** 方程 $A^T P + PA + Q - PBR^{-1}B^T P = 0$ 的解。

类似的, 对于离散线性时不变系统 (假设系统完全能控), 设其状态方程为

$$X(k+1) = AX(k) + BU(k) \quad (17)$$

二次型性能指标为

$$J = \sum_{k=0}^{+\infty} [X^T(k)QX(k) + U^T(k)RU(k)] \quad (18)$$

式中, Q 为对称正定的常数阵或对称半正定的常数阵, R 为对称正定的常数阵。

现在的问题是设计一个状态反馈控制器 $U(k) = -LX(k)$, 使得 J 最小。和上面一样, 这里只给出相应结论:

$$L = (R + B^T P B)^{-1} B^T P A \quad (19)$$

其中, P 是 **Riccati** 方程 $Q - P + A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A = 0$ 的解。

3.2 实例

有了上述对 LQR 控制器定性的说明后, 接下来以四旋翼无人机的姿态控制为实例说明 LQR 控制器的设计。通常情况下, 我们并不会手算状态反馈矩阵, 而是借用 MATLAB 的相关函数进行计算, 相关程序如下所示:

```
clc
clear
close all
%% Parameter
m1 = 0.079 + 0.060; % 单位: kg
m2 = 0.014; % 单位: kg
m = m1 + 4*m2; % 单位: kg
I_xx = (1/12)*m1*(0.069^2+0.012^2) + 4*m2*0.0636^2; % 单位: kg·m²
I_yy = (1/12)*m1*(0.069^2+0.012^2) + 4*m2*0.0636^2; % 单位: kg·m²
I_zz = (1/12)*m1*(0.069^2+0.069^2) + 4*m2*0.0900^2; % 单位: kg·m²
%% Displacement (X-axis)
A = [0 1; 0 0];
B = [0; 9.8];
C = eye(2);
D = 0;
Ts = 0.005;
```

```

t = 0:Ts:10;
u = zeros(size(t));
[G,H] = c2d(A,B,Ts); % discretizing
x0 = [-1; 0];
Tc = ctrb(G,H);
if (rank(Tc)==2)
    Q = [5 0; 0 10];
    R = 1e2;
    Kx = dlqr(G,H,Q,R);
    G2 = G-H*Kx;
    H = zeros(2,1);
    y = dlsim(G2,H,C,D,u,x0);
    subplot(3,1,1)
    plot(t,y(:,1),'b','LineWidth',1.5);
    xlabel('Time(s)');
    ylabel('x(m)');
    grid on
end
%% Displacement(Y-axis)
A = [0 1; 0 0];
B = [0; -9.8];
C = eye(2);
D = 0;
Ts = 0.005;
t = 0:Ts:10;
u = zeros(size(t));
[G,H] = c2d(A,B,Ts); % discretizing
x0 = [-1; 0];
Tc = ctrb(G,H);
if (rank(Tc)==2)
    Q = [5 0; 0 10];
    R = 1e2;
    Ky = dlqr(G,H,Q,R);
    G2 = G-H*Ky;
    H = zeros(2,1);
    y = dlsim(G2,H,C,D,u,x0);
    subplot(3,1,2)
    plot(t,y(:,1),'b','LineWidth',1.5);
    xlabel('Time(s)');
    ylabel('y(m)');
    grid on
end
%% Height
A = [0 1; 0 0];

```

```

B = [0; 1/m];
C = eye(2);
D = 0;
Ts = 0.005;
t = 0:Ts:10;
u = zeros(size(t));
[G,H] = c2d(A,B,Ts); % discretizing
x0 = [-1; 0];
Tc = ctrb(G,H);
if (rank(Tc)==2)
    Q = [30 0; 0 40];
    R = 1e2;
    K1 = dlqr(G,H,Q,R);
    G2 = G-H*K1;
    y = dlsim(G2,H,C,D,u,x0);
    subplot(3,1,3)
    plot(t,y(:,1),'b','LineWidth',1.5);
    xlabel('Time(s)');
    ylabel('h(m)');
    grid on
end

%% Roll
A = [0 1; 0 0];
B = [0; 1/I_xx];
C = eye(2);
D = 0;
Ts = 0.005;
t = 0:Ts:10;
u = zeros(size(t));
[G,H] = c2d(A,B,Ts); % discretizing
x0 = [0.0873; 0];
Tc = ctrb(G,H);
if (rank(Tc)==2)
    Q = [500 0; 0 10];
    R = 5e4;
    K2 = dlqr(G,H,Q,R);
    G2 = G-H*K2;
    y = dlsim(G2,H,C,D,u,x0);
    figure
    subplot(3,1,1)
    plot(t,y(:,1),'b','LineWidth',1.5);
    xlabel('Time(s)');
    ylabel('\phi(rad)');
    grid on

```

```

end
%% Pitch
A = [0 1; 0 0];
B = [0; 1/I_yy];
C = eye(2);
D = 0;
Ts = 0.005;
t = 0:Ts:10;
u = zeros(size(t));
[G,H] = c2d(A,B,Ts); % discretizing
x0 = [0.0873; 0];
Tc = ctrb(G,H);
if (rank(Tc)==2)
    Q = [500 0; 0 10];
    R = 5e4;
    K3 = dlqr(G,H,Q,R);
    G2 = G-H*K3;
    y = dlsim(G2,H,C,D,u,x0);
    subplot(3,1,2)
    plot(t,y(:,1),'b','LineWidth',1.5);
    xlabel('Time(s)');
    ylabel("\theta(rad)");
    grid on
end
%% Yaw
A = [0 1; 0 0];
B = [0; 1/I_zz];
C = eye(2);
D = 0;
Ts = 0.005;
t = 0:Ts:10;
u = zeros(size(t));
[G,H] = c2d(A,B,Ts); % discretizing
x0 = [0.0873; 0];
Tc = ctrb(G,H);
if (rank(Tc)==2)
    Q = [100 0; 0 10];
    R = 1e5;
    K4 = dlqr(G,H,Q,R);
    G2 = G-H*K4;
    y = dlsim(G2,H,C,D,u,x0);
    subplot(3,1,3)
    plot(t,y(:,1),'b','LineWidth',1.5);
    xlabel('Time(s)');

```

```

ylabel("\psi (rad)");
grid on
end

```

运行后的结果如下所示：

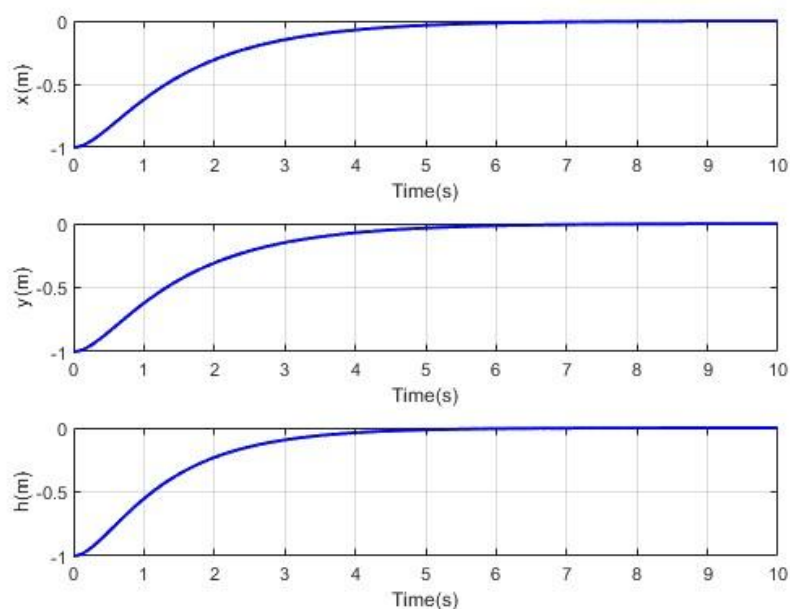


图 3-2 离散时间系统 LQR 仿真结果（位置）

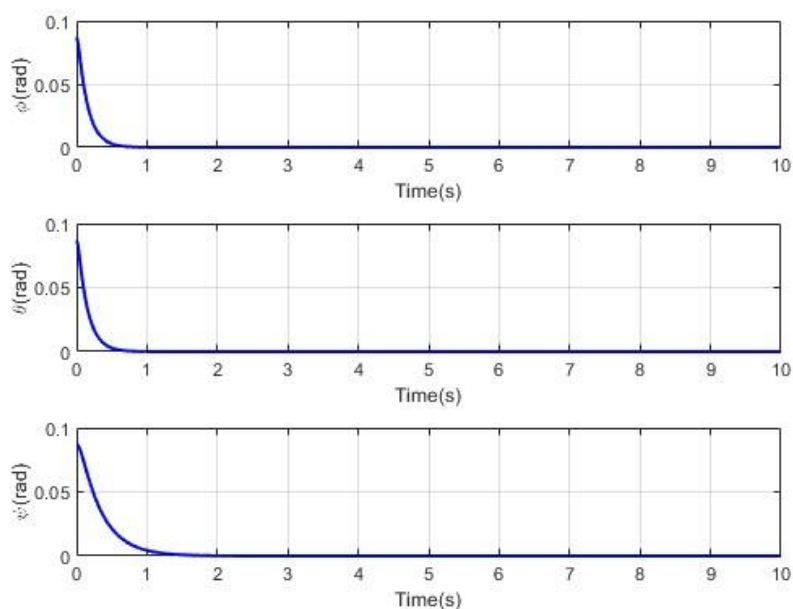


图 3-3 离散时间系统 LQR 仿真结果（姿态）

值得注意的是，系统的实际输入是 4 个旋翼的升力，考虑到如下关系式

$$\begin{cases} f_i = C_T \omega_i^2 \\ C_i = C_M \omega_i^2 \end{cases} \quad (20)$$

则系统的实际输入可表示成

$$\begin{bmatrix} u_1 + mg \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}}b & \frac{1}{\sqrt{2}}b & \frac{1}{\sqrt{2}}b & -\frac{1}{\sqrt{2}}b \\ -\frac{\sqrt{2}}{2}b & \frac{\sqrt{2}}{2}b & \frac{\sqrt{2}}{2}b & \frac{\sqrt{2}}{2}b \\ -\frac{\sqrt{2}}{2}b & -\frac{\sqrt{2}}{2}b & \frac{\sqrt{2}}{2}b & \frac{\sqrt{2}}{2}b \\ \frac{C_M}{C_T} & -\frac{C_M}{C_T} & \frac{C_M}{C_T} & -\frac{C_M}{C_T} \end{bmatrix} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix} \quad (21)$$

即

$$\begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix} = \begin{bmatrix} \frac{1}{4} & -\frac{\sqrt{2}}{4b} & -\frac{\sqrt{2}}{4b} & \frac{C_T}{4C_M} \\ \frac{1}{4} & \frac{\sqrt{2}}{4b} & -\frac{\sqrt{2}}{4b} & -\frac{C_T}{4C_M} \\ \frac{1}{4} & \frac{\sqrt{2}}{4b} & \frac{\sqrt{2}}{4b} & \frac{C_T}{4C_M} \\ \frac{1}{4} & -\frac{\sqrt{2}}{4b} & \frac{\sqrt{2}}{4b} & -\frac{C_T}{4C_M} \end{bmatrix} \begin{bmatrix} u_1 + mg \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} \quad (22)$$

对应的 C 语言程序为

```
// 位置反馈系数
static const float kp = 0.2215f;
static const float kd = 0.3786f;

// 状态反馈系数
static const float k11 = 0.5422f;
static const float k12 = 0.7769f;
static const float k21 = 0.0868f;
static const float k22 = 0.0141f;
static const float k31 = 0.0868f;
static const float k32 = 0.0141f;
static const float k41 = 0.0300f;
static const float k42 = 0.0111f;
...
// 姿态控制器（包含了高度控制）
static void Attitude_Controller(const ATTITUDE_DATA_t* attitude, const IMU_DATA_t*
imudata, const float Height, const float Height_dot, const float LidarPos[], const
float LidarPos_dot[], const float Ts, float motorf[4][1])
{
    float u[4][1] = {0}; // 系统的输入量
    float T[4][4] =
```



```

        {0.25f, -3.9284f, -3.9284f, 50.2828f},
        {0.25f, 3.9284f, -3.9284f, -50.2828f},
        {0.25f, 3.9284f, 3.9284f, 50.2828f},
        {0.25f, -3.9284f, 3.9284f, -50.2828f}};

...

float pitch_ref = -(kp*x + kd*x_dot); // 位置 LQR 控制器

...

float roll_ref = kp*y + kd*y_dot; // 位置 LQR 控制器

...

// LQR 控制器
u[0][0] = -(k11 * (Height - FlyControl_height) + k12 * Height_dot) +
height_to_weight(FlyControl_height) + FlyControl_unuseHeight;
u[1][0] = -(k21 * (attitude -> roll + FlyControl_roll - roll_ref) + k22 *
imudata -> gyro.x);
u[2][0] = -(k31 * (attitude -> pitch + FlyControl_pitch - pitch_ref) + k32 *
imudata -> gyro.y);
u[3][0] = -(k41 * (attitude -> yaw - FlyControl_yaw) + k42 * (imudata -> gyro.z
+ FlyControl_gyroz));

...

// 系统的输入转换为 4 个电机的升力
mul(4, 4, 4, 1, (const float (*)[4])T, (const float (*)[1])u, motorf);
}

```

4. PID 控制器

4.1 控制方案与原理

在 PID 控制器中，不需要对四轴进行建模分析，只需要知道四轴的翻滚、旋转、上升与下降如何控制即可。详情介绍请查看图 2-1 以及相应的说明。

四轴飞行器采用串级 PID 控制，其控制结构如下：外环控制器为角度环，输入为目标角度和实际角度，输出为角速度的目标值。外环特点是响应较慢，更侧重于整体的系统稳定，抑制累计误差等。内环控制器为角速度环，输入为外环的输出值和实际的角速度，输出为电机的油门值，其特点是响应快速，增加系统的抗干扰能力。整体的系统模型如下图所示：

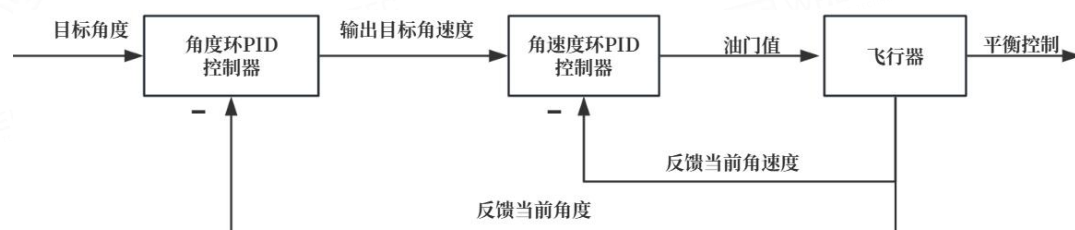
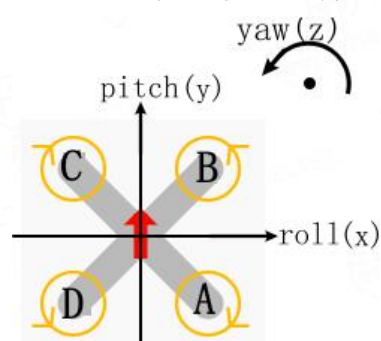


图 4-1 串级 PID 控制器模型

有了串级 PID 控制器后，还需要对 PID 控制器输出整合到电机油门，完成最终的控制。四轴的模型如下图所示（陀螺仪方向符合右手定则）：



假设飞行器向前翻滚，为了纠正飞行器，则需要将 B、C 电机提高油门，A、D 电机降低油门；向后反之。假设飞行器逆时针偏航，则需要将电机 A、C 降低油门，B、D 提高油门；顺时针反之。通过这个控制原理可推出最终的电机输出整合方案如下（在整合时，需要特别注意 PID 输出的方向是否正确）：

$$\begin{aligned} MA &= -RollRate - PitchRate + YawRate \\ MB &= +RollRate - PitchRate - YawRate \\ MC &= +RollRate + PitchRate + YawRate \\ MD &= -RollRate + PitchRate - YawRate \end{aligned}$$

除了整合 PID 的输出外，电机还需要有基础油门才能维持平衡，基础油门的大小跟机身重量相关。可使用附录中的油门与推力关系推出推力大小与油门的关系。在 PID 控制版本的代码中，将油门与推力的关系拟合成了如下公式，其中重量单位是 g：

其中这里还包含了高度与重量的关系，因为飞行器底下拉着电源线，电源线每米 14g，所以有 $height \times 14.0f$ 的关系。

```
//重量转为油门值,入口参数单位克 g
static uint16_t weight_to_throttle(float weight,float height)
{
    weight = weight + height*14.0f;
    return 156 + 6.8f*weight - 0.0232f*(weight*weight) +
0.0000512f*(weight*weight*weight);
}
```

除了基础油门外，要使得飞行器能稳定在一定高度上，则需要加入激光测距模块以及高度的串级 PID 控制，控制模型仍是图 4-1 所述。

综合上述，在四轴飞行器的 PID 控制器中，有基础油门、3 轴角度环、高度环的控制，最终整合到油门如下所示：

```
//A 电机油门
motor.A.throttle = base_throttle - RollRatePID.output - PitchRatePID.output +
YawRatePID.output + HeightSpeedPID.output;

//B 电机油门
motor.B.throttle = base_throttle + RollRatePID.output - PitchRatePID.output -
YawRatePID.output + HeightSpeedPID.output;

//C 电机油门
motor.C.throttle = base_throttle + RollRatePID.output + PitchRatePID.output +
YawRatePID.output + HeightSpeedPID.output;

//D 电机油门
motor.D.throttle = base_throttle - RollRatePID.output + PitchRatePID.output -
YawRatePID.output + HeightSpeedPID.output;
```

4.2 PID 参数调试方法介绍

一般在串级的 PID 参数调试中，先整定内环，再整定外环。内环的响应要求比外环快，且内环是外环的基础，当内环参数足够稳定时，可使得外环的输入快速响应，最终到达稳定的效果。在调试 PID 参数时，可将飞行器绑定在调试

架上单独对 roll 或者 pitch 环进行调试。以下是调参经验的参考：

内环 K_p 调参参考：只有 K_p 参数时，可以让飞行器在调试架上缓慢掉落且不抖动。

内环 K_d 调参参考：可根据 K_p 的情况调整，如果出现抖动，加入较小的 K_d 参数解决抖动问题，若没有明显的抖动， k_d 参数有时也可以不加，但是抗干扰性会差一些。

内环 K_i 调参参考：调整完 k_p 、 k_i 后，尝试加入 k_i 项，当加入合适的 k_i 时，飞行器可以稳定在调试架上不掉落，稳定的角度不固定，因为此时没有加入外环。

外环 K_p 调参参考：加入合适的 K_p 后，飞行器能稳定在目标角度。

外环 K_d 调参参考：加入合适的 K_d 后，飞行器具有一定的抗干扰能力，用手拨动可感觉到飞行器反抗明显，并快速回到目标角度。 K_d 值能增加抗干扰能力，但是不宜过大，否则飞行器容易抖动并伴随电机发热。

外环 K_i 调参参考：若飞行器无法到达目标角度，可适当增加 k_i 的值，若飞行器比较稳定没有误差，则不需要增加 k_i 项。

在调整完 pid 参数后，可使用手柄对飞行器输入目标角度，并频繁切换目标值，比如 $-10^\circ \sim 10^\circ$ 的目标值来换切换，如果飞行器能稳定跟随且不发散，则参数基本调节完毕。若追求更好的参数效果，还可以通过检测整机电流的方式，找到一组飞行器稳定后，整机电流最小的参数。

5. 附录

5.1 升力与 PWM 值的关系

表 4-1 PWM-Thrust

PWM	Thrust / g (4 个)	Thrust / g (1 个)	Thrust / N (1 个)
150	3	0.75	0.00735
200	7	1.75	0.01715
250	13	3.25	0.03185
300	21	5.25	0.05145
350	30	7.5	0.0735
400	40	10	0.098
450	50	12.5	0.1225
500	61	15.25	0.14945
550	73	18.25	0.17885
600	86	21.5	0.2107
650	100	25	0.245
700	116	29	0.2842
750	128	32	0.3136
800	143	35.75	0.35035
850	160	40	0.392
900	176	44	0.4312
950	186	46.5	0.4557
1000	200	50	0.49
1050	211	52.75	0.51695
1100	223	55.75	0.54635
1150	236	59	0.5782
1200	248	62	0.6076
1250	260	65	0.637

1300	270	67.5	0.6615
1350	279	69.75	0.68355
1400	288	72	0.7056
1450	294	73.5	0.7203
1500	300	75	0.735

拟合后的关系式如下图所示：

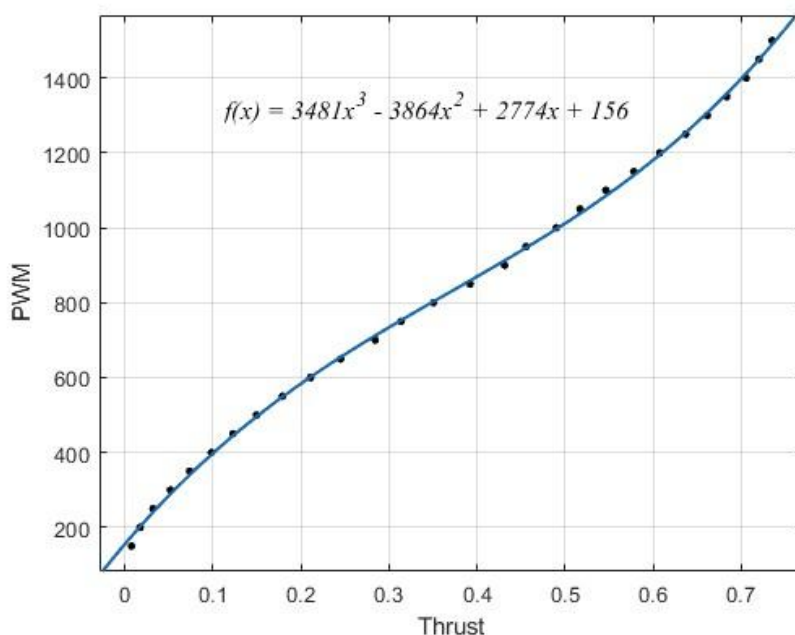


图 4-1 PWM-Thrust 关系曲线拟合结果

5.2 四元数的求导公式

定义四元数 $q(t)$ 来表示 t 时刻无人机从世界坐标系 E 到机体坐标系 B 的变换关系，在 $t + \Delta t$ 时刻它们之间的变换关系表示为 $q(t + \Delta t)$ ，则在 Δt 这个时间段内微小的旋转变化为 $q(t + \Delta t) - q(t)$ 。若设这个微小旋转的瞬时旋转角速度为 ω ，转过的角度大小为 $\Delta\theta$ ，则有

$$\Delta\theta = \|\omega\|\Delta t$$

以及归一化的角速度：

$$\hat{\omega} = \frac{\omega}{\|\omega\|}$$

那么这个微小的旋转变化可以由四元数表示为

$$\Delta q = \left[\cos \frac{\Delta \theta}{2}, \sin \frac{\Delta \theta}{2} \hat{\omega} \right] = \left[\cos \frac{\|\omega\| \Delta t}{2}, \sin \frac{\|\omega\| \Delta t}{2} \hat{\omega} \right]$$

设初始旋转为 $q(t) = [s, \mathbf{v}]$ ，则有

$$q(t + \Delta t)$$

$$= q \otimes \Delta q$$

$$= \left[s \cos \frac{\|\omega\| \Delta t}{2} - \mathbf{v}^T \sin \frac{\|\omega\| \Delta t}{2} \hat{\omega}, \mathbf{v} \cos \frac{\|\omega\| \Delta t}{2} + s \sin \frac{\|\omega\| \Delta t}{2} \hat{\omega} + \mathbf{v} \times \sin \frac{\|\omega\| \Delta t}{2} \hat{\omega} \right]$$

所以

$$\dot{q}$$

$$= \lim_{\Delta t \rightarrow 0} \frac{q(t + \Delta t) - q(t)}{\Delta t}$$

$$= \lim_{\Delta t \rightarrow 0} \frac{\left[\left(\cos \frac{\|\omega\| \Delta t}{2} - 1 \right) s - \mathbf{v}^T \sin \frac{\|\omega\| \Delta t}{2} \hat{\omega}, \left(\cos \frac{\|\omega\| \Delta t}{2} - 1 \right) \mathbf{v} + s \sin \frac{\|\omega\| \Delta t}{2} \hat{\omega} + \mathbf{v} \times \sin \frac{\|\omega\| \Delta t}{2} \hat{\omega} \right]}{\Delta t}$$

$$= \left[-\mathbf{v}^T \frac{\|\omega\|}{2} \hat{\omega}, s \frac{\|\omega\|}{2} \hat{\omega} + \mathbf{v} \times \frac{\|\omega\|}{2} \hat{\omega} \right]$$

$$= \frac{1}{2} [-\mathbf{v}^T \boldsymbol{\omega}, s \boldsymbol{\omega} + \mathbf{v} \times \boldsymbol{\omega}]$$

$$= \frac{1}{2} q \otimes [0, \boldsymbol{\omega}]$$

即

$$\dot{q} = \frac{1}{2} q \otimes [0, \boldsymbol{\omega}]$$

5.3 四元数与欧拉角的关系

从世界坐标系 E 到机体坐标系 B 的旋转矩阵可以用四元数表示为：

$$R = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1 q_2 - q_0 q_3) & 2(q_1 q_3 + q_0 q_2) \\ 2(q_1 q_2 + q_0 q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2 q_3 - q_0 q_1) \\ 2(q_1 q_3 - q_0 q_2) & 2(q_2 q_3 + q_0 q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

注： $q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1$ 。

如果用欧拉角来表示则可以表示为

$$R = \begin{bmatrix} \cos \psi \cos \theta & \cos \psi \sin \theta \sin \varphi - \sin \psi \cos \varphi & \cos \psi \sin \theta \cos \varphi + \sin \psi \sin \varphi \\ \sin \psi \cos \theta & \sin \psi \sin \theta \sin \varphi + \cos \psi \cos \varphi & \sin \psi \sin \theta \cos \varphi - \cos \psi \sin \varphi \\ -\sin \theta & \sin \varphi \cos \theta & \cos \varphi \cos \theta \end{bmatrix}$$

通过对比可得

$$\begin{cases} \varphi = \text{atan2}(2(q_2q_3 + q_0q_1), q_0^2 - q_1^2 - q_2^2 + q_3^2) \\ \theta = -\text{asin}(2(q_1q_3 - q_0q_2)) \\ \psi = \text{atan2}(2(q_1q_2 + q_0q_3), q_0^2 + q_1^2 - q_2^2 - q_3^2) \end{cases}$$

5.4 显式互补滤波(ECF)

众所周知，陀螺仪短时间内可提供高精度的姿态数据，但因漂移的影响，长时间后，精度就会降低。加速度计测量误差不会随时间累计，但其无法区分重力加速度和运动加速度，短期内的精度差。因此，根据两者在频域上的互补特性，利用互补滤波器对两者进行数据融合可以提高姿态测量精度。显式互补滤波具有计算量小，容易实现的优点，是目前低端处理器场合首选的滤波方法。

首先将 $\dot{q} = \frac{1}{2} \mathbf{q} \otimes [0, \boldsymbol{\omega}]$ 写成矩阵形式：

$$\begin{bmatrix} \dot{q}_0 \\ \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}$$

另一方面，我们知道

$$\mathbf{q}(t + \Delta t) = \mathbf{q}(t) + \dot{\mathbf{q}}(t)\Delta t$$

若初始时刻 $\mathbf{q}(0)$ 是准确的（这里只是作为一个简单的理解，实际应用中对初始值没有要求），那么造成后面姿态 $\mathbf{q}(t)$ 不准确的原因只可能是 $\dot{\mathbf{q}}(t)$ ，进一步的，因为 $\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes [0, \boldsymbol{\omega}]$ ，所以说造成后面姿态 $\mathbf{q}(t)$ 不准确的原因是角速度 $\boldsymbol{\omega}$ 的不准确导致的，为此我们可以对角速度进行补偿来修正机体的姿态。在互补滤波中，这个补偿量由加速度计来提供。

加速度计在机体静止状态下的理想测量值为 $\mathbf{v} = [0 \ 0 \ -g]^T$ ，进行归一化后有 $\hat{\mathbf{v}} = [0 \ 0 \ -1]^T$ 。当机体发生倾斜时，加速度计的理想测量值应该为

$$\hat{\mathbf{v}} = R \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 2(q_1q_3 + q_0q_2) \\ 2(q_2q_3 - q_0q_1) \\ -q_0^2 + q_1^2 + q_2^2 - q_3^2 \end{bmatrix}$$

设机体的实际测量值为 $\mathbf{a} = [a_x \ a_y \ a_z]^T$ ，则归一化后的测量值为 $\hat{\mathbf{a}} = \mathbf{a}/\|\mathbf{a}\|$ 。

定义误差向量为 $\hat{\mathbf{e}} = \hat{\mathbf{a}} \times \hat{\mathbf{v}}$ ，叉积的方向与两向量垂直，叉积的模正比于两向量夹角的正弦值，误差向量表示将估计值得到的向量旋转成与加速度计输出得到的向量平行所需要的旋转方向和旋转量。接下来，将误差向量通过一个 PI 控制器与陀螺仪的输出进行互补滤波，即

$$\begin{cases} \dot{q} = \frac{1}{2}q \otimes [0, \omega + \delta] \\ \delta = k_p \hat{e} + k_i \sum \hat{e} \\ q(t + \Delta t) = q(t) + \dot{q}(t)\Delta t \end{cases}$$

5.5 前馈补偿器

在我们的配件中，无刷电机是一种开环控制系统（既没有速度传感器也没有力矩传感器），为了易于分析，用一个简单的纯延时系统来作为它的近似模型，如下图所示：

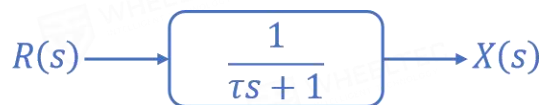


图 4-2 系统传递函数

其中， $R(s)$ 为目标力矩， $X(s)$ 为实际力矩， τ 为延迟时间。

由于无法通过反馈来进行力矩控制，为了能减少时滞所造成的影响，加入一个前馈补偿器以改善系统的响应速度，如下图所示：

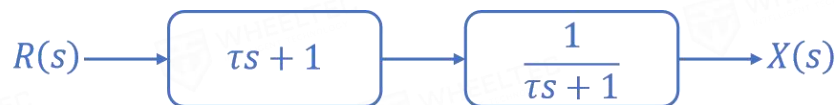


图 4-3 加入前馈补偿器后的系统传递函数

即目标力矩经过一个类似于“PD 控制器”结构的系统后再给到无刷电机（ τ 根据实际情况进行参数整定）。对应的 C 语言程序为

```
// 前馈补偿器
static float u10Prev = 0, u20Prev = 0, u30Prev = 0;
u[1][0] += (u[1][0] - u10Prev)*0.0015f;
u[2][0] += (u[2][0] - u20Prev)*0.0015f;
u[3][0] += (u[3][0] - u30Prev)*0.0015f;
u10Prev = u[1][0];
u20Prev = u[2][0];
u30Prev = u[3][0];
```

5.6 高通滤波器

高通滤波器，又称低阻滤波器，允许高于某一截频的频率通过，而大大衰减较低频率的一种滤波器。它去掉了信号中不必要的低频成分或者说去掉了低频干扰。

考虑下面这个电路系统：

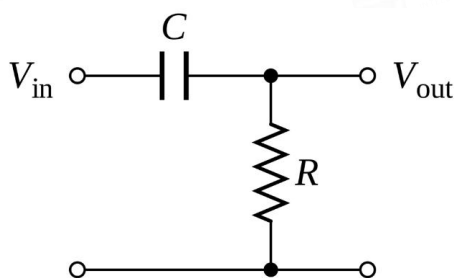


图 4-4 高通滤波器电路

由电路原理容易写出它的方程：

$$\begin{cases} u_o = R \cdot i(t) \\ i(t) = C \cdot \frac{du_C}{dt} \\ u_i = u_C + u_o \end{cases}$$

其中， u_C 为电容两端的电压， u_i 为输入电压， u_o 为输出电压。对其做拉普拉斯变换可得系统的传递函数为

$$H(s) = \frac{U_o(s)}{U_i(s)} = \frac{s}{\frac{1}{RC} + s}$$

其伯德图（假设 $RC = 0.01$ ）如图 4-5 所示。由图可见，系统中信号低频的部分被衰减，高频的部分被保留，有效的去掉了信号中不必要的低频成分，所以该系统是一个高通滤波器。

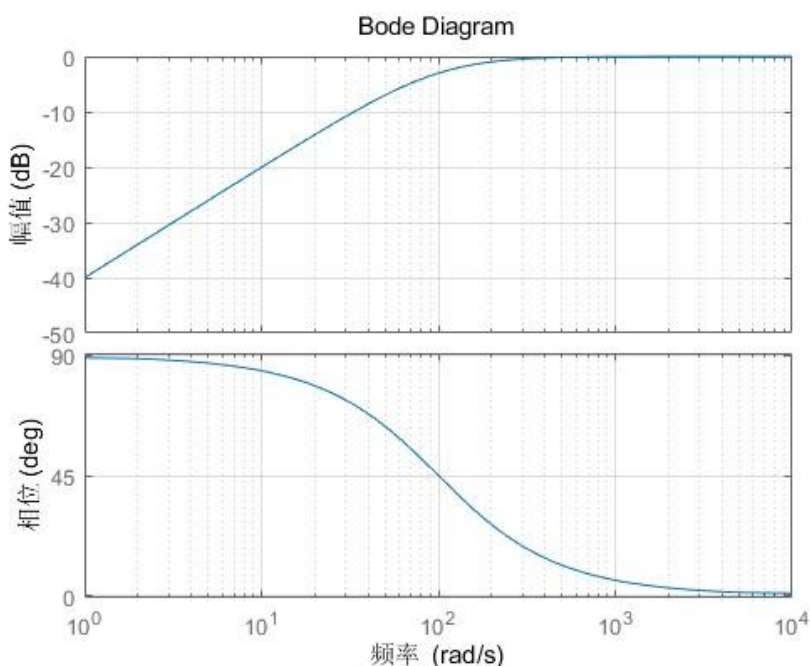


图 4-5 高通滤波器电路的伯德图

下面求解一阶高通滤波器的离散形式：采用一阶后向差分来近似微分，可以得到：

$$u_o(k) = R \cdot i(t) = RC \cdot \left(\frac{u_i(k) - u_i(k-1)}{T_s} - \frac{u_o(k) - u_o(k-1)}{T_s} \right)$$

其中， T_s 为采样周期，对上式进行整理有

$$u_o(k) = \frac{RC}{T_s + RC} \cdot (u_i(k) - u_i(k-1) + u_o(k-1))$$

令

$$\alpha = \frac{RC}{T_s + RC}$$

故

$$u_o(k) = \alpha \cdot (u_i(k) - u_i(k-1) + u_o(k-1))$$

以上就是一阶高通滤波器的离散形式，对应的 C 语言代码如下所示：

// 一阶高通滤波器

```
static float HPFofXdot(float x, const float alpha, uint8_t resetFlag)
{
    static float x_prev = 0;
    static float y_prev = 0;
    if (resetFlag == 1) x_prev = y_prev = 0;
    float y = alpha*(y_prev + x - x_prev);
    x_prev = x;
    y_prev = y;
    return y;
}
```

5.7 雷达定位算法

前提假设：

- (1) 假设机体的偏航角 φ 变化很小（即假设机体偏航方向的控制做得比较好），忽略不计；
- (2) 假设机体的运动速度不太快以至于可以忽略雷达点云的畸变。

算法步骤：

- **STEP 1:**

初始化初始位置

$$\begin{cases} x_0 = 0 \\ y_0 = 0 \end{cases}$$

● STEP 2:

在机体开始运动之前，先采集一圈稳定的点云数据作为模板点云数据，数据格式为 (r, θ) ，其中， r 为距离， θ 为雷达扫描角度。

● STEP 3:

数据预处理，包括：

- 1) 滤除距离太近（小于 3cm）和距离太远（超过扫描距离时 $r = 0$ ）的点。
- 2) 通过下式将极坐标转换为直角坐标：

$$\begin{cases} x_i^0 = r_i^0 \sin \theta_i^0, i = 1 \cdots N \\ y_i^0 = r_i^0 \cos \theta_i^0, i = 1 \cdots N \end{cases}$$

其中，上标“0”表示模板数据，若是目标数据则用上标“1”表示以示区分，下标“ i ”表示第 i 个点云数据。

- 3) 数据去中心化处理：

$$\begin{cases} \bar{x}^0 = \frac{\sum_{i=1}^N x_i^0}{N} \\ \bar{y}^0 = \frac{\sum_{i=1}^N y_i^0}{N} \\ \widehat{x}_i^0 = x_i^0 - \bar{x}^0, i = 1 \cdots N \\ \widehat{y}_i^0 = y_i^0 - \bar{y}^0, i = 1 \cdots N \end{cases}$$

● STEP 4:

将二维数组 $(\widehat{x}_i^0, \widehat{y}_i^0)$ 以 KD - Tree 的形式存储起来。其中 KD - Tree 的基本构建流程如下：

- 1) 选择分割维度：从所有维度中选择方差最大的维度作为分割维度（或者自己预设一个初始分割维度）；
- 2) 选择中位数：在该维度上选择数据点的中位数作为分割点；
- 3) 分割数据：根据分割维度将数据点分为两部分，分别递归构建左右子树；
- 4) 递归构建：对左右子树重复上述过程，直到每个叶子节点只包含一个数据点。

● STEP 5:

在机体开始运动后，采集到新的一圈点云数据作为目标点云数据，经过预处理

理后（滤除不满足条件的点，转换成直角坐标以及去中心化）的数据为

$$\begin{cases} \overline{x^1} = \frac{\sum_{i=1}^N x_i^1}{N} \\ \overline{y^1} = \frac{\sum_{i=1}^N y_i^1}{N} \\ \widehat{x_i^1} = x_i^1 - \overline{x^1}, i = 1 \cdots N \\ \widehat{y_i^1} = y_i^1 - \overline{y^1}, i = 1 \cdots N \end{cases}$$

● **STEP 6:**

初始化位移估值

$$\begin{cases} \Delta x = \overline{x^0} - \overline{x^1} \\ \Delta y = \overline{y^0} - \overline{y^1} \end{cases}$$

● **STEP 7:**

接着，将目标点云数据和模板点云数据进行匹配，找到目标点云数据中的每一个点与模板点云数据的最短距离 d_i （为了减少计算量，这里采用曼哈顿距离），得到对应的两组最短距离数组 $\{dx_i\}, \{dy_i\}, i = 1 \cdots N$ （ x 方向上的和 y 方向上的），其中 $d_i = dx_i + dy_i$ 。用距离数组的中位数更新位移估值：

$$\begin{cases} \Delta x = \Delta x - \text{med}(\{dx_i\}) \\ \Delta y = \Delta y - \text{med}(\{dy_i\}) \end{cases}$$

并计算最大偏移量（定义最大偏移量为两个方向偏移量的绝对值之和）

$$D = |\Delta x| + |\Delta y|$$

● **STEP 8:**

若最大偏移量小于设置的阈值（我们设置为 0.1），则跳转至 **STEP 11**。否则继续以下步骤。

● **STEP 9:**

更新目标点云数据：

$$\begin{cases} \widehat{x_i^1} = \widehat{x_i^1} - \Delta x \\ \widehat{y_i^1} = \widehat{y_i^1} - \Delta y \end{cases}$$

● **STEP 10:**

回到 **STEP 7**，直到满足最大偏移量小于设置的阈值。

- **STEP 11:**

更新机体的位置:

$$\begin{cases} x = x_0 + \Delta x \\ y = y_0 + \Delta y \end{cases}$$

- **STEP 12:**

当任意一个方向上的位移估值达到阈值（我们设置为 500mm）的时候，记录当前位置，把当前的位置作为新的坐标原点并同时更新模板点云数据（回到 **STEP 3**）：

$$\begin{cases} x_0 = x \\ y_0 = y \end{cases}$$

否则直接回到 **STEP 5**，对下一圈数据进行处理。

上述算法的流程图如下所示：

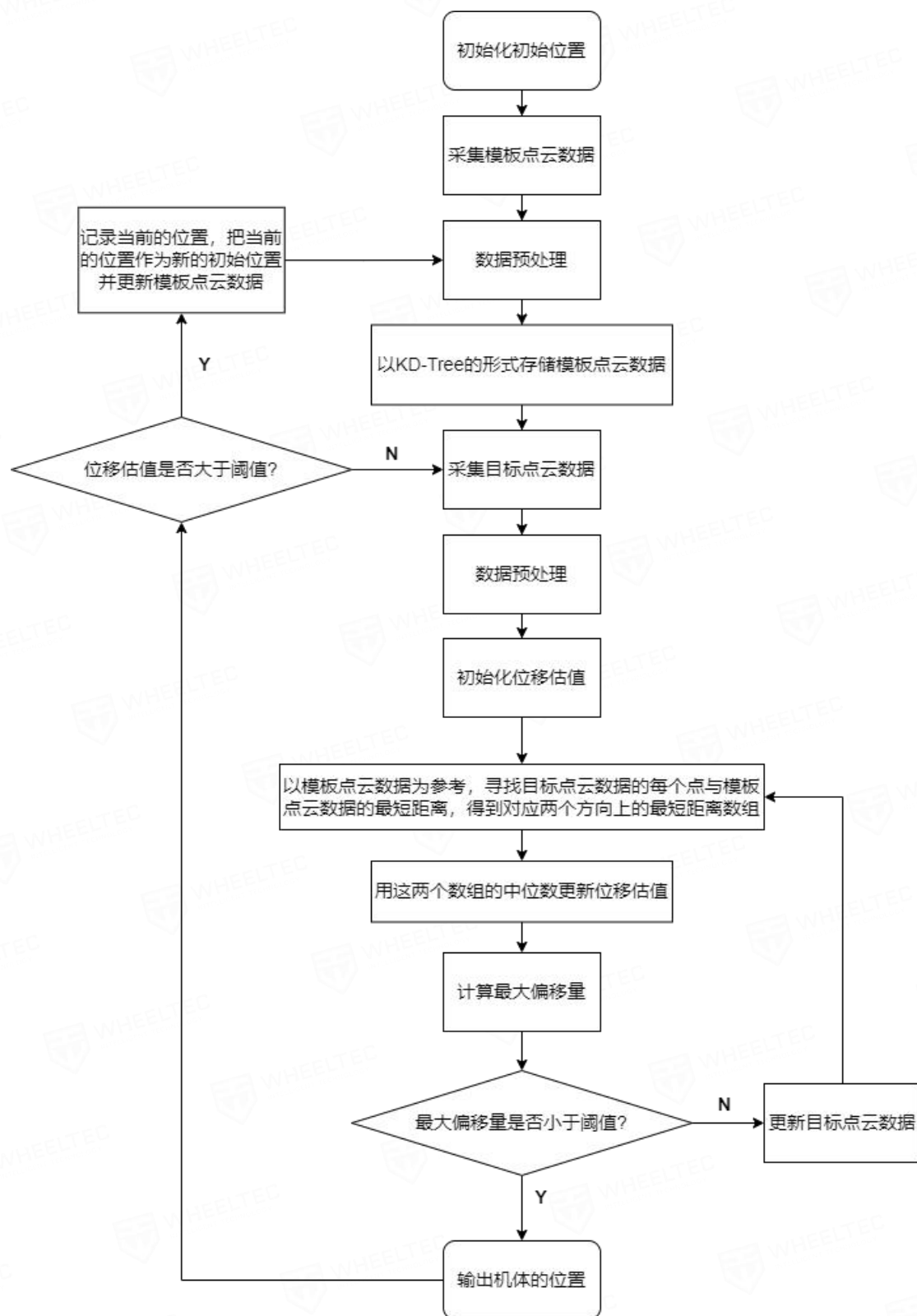


图 4-6 程序流程图

5.8 光流数据处理

光流这个概念源于生物视觉，例如当一只苍蝇飞过，青蛙能迅速捕捉到它的运动，这背后就是光流原理在起作用。在技术领域，光流定义为“观察者和场景之间的相对运动在图像平面上产生的瞬时速度场”。简单来说，它就是通过分析连续的图像帧，计算出图像中每个像素点的运动速度和方向，从而描绘出一幅“运动矢量场”。基于这个特点所制作的光流模块在无 GPS 环境下，可以实时检测无人机水平移动距离，从而实现对无人机的悬停控制。

● 旋转补偿

因为光流模块固连在无人机底部，无人机倾斜会造成光流模块拍摄的图像产生平移，这时候产生的光流数据读数并不能真实反映无人机的运动，需要进行误差补偿。由机体旋转所造成的光流变化正负及具体补偿方式需要根据具体坐标系进行判断。

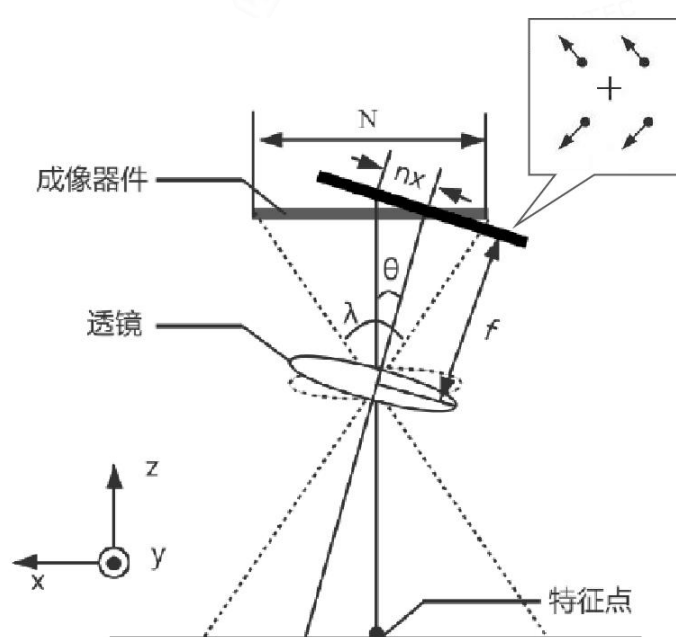


图 4-7 旋转补偿原理图

图 4-7 所示为无人机绕 Y 轴旋转角度后光流的变化。其中， N 为光流传感器内部成像器件在 x 方向上的像素点个数， λ 为视场角， f 为微型摄像头的焦距。当无人机绕旋转角度 θ 后，光流传感器所研究的特征点在成像器件上移动了 n_x 个像素点，由几何对应关系，与具体关系如下：

$$n_x = -\frac{N}{2 \tan \frac{\lambda}{2}} \tan \theta = -f \tan \theta$$

同理，绕着 X 轴进行旋转一个角度 φ ， n_y 的推导公式类似。当 θ 和 φ 都取小角度的时候，由极限的等价无穷小可以将公式化简如下：

$$\begin{cases} n_x = \frac{N}{2 \tan \frac{\lambda}{2}} = -f \cdot \theta \\ n_y = \frac{N}{2 \tan \frac{\lambda}{2}} = f \cdot \varphi \end{cases}$$

从光流传感器获取到单位时间内的位移输出后，就可以利用 θ 和 φ 的变化率即陀螺仪输出的角速度对光流传感器数据进行补偿，抵消无人机倾斜对光流传感器数据造成的影响，即：

$$\begin{cases} \dot{x} \approx \frac{\Delta N_x - \Delta n_x}{f \Delta t} = \frac{\Delta N_x - \Delta n_x}{f \Delta t} = \frac{\Delta N_x}{f \Delta t} + \dot{\theta} \\ \dot{y} \approx \frac{\Delta N_y - \Delta n_y}{f \Delta t} = \frac{\Delta N_y - \Delta n_y}{f \Delta t} = \frac{\Delta N_y}{f \Delta t} - \dot{\varphi} \end{cases}$$

其中， ΔN_x 和 ΔN_y 是实际的光流模块的输出值。

注：旋转补偿往往要求陀螺仪数据与光流数据相位对齐，即当机体只有旋转运动的时候，经过旋转补偿后的数据接近 0。下面是只有旋转运动的时候用上位机采集到的数据曲线（红色曲线为光流原始数据，紫红色曲线为陀螺仪数据）：

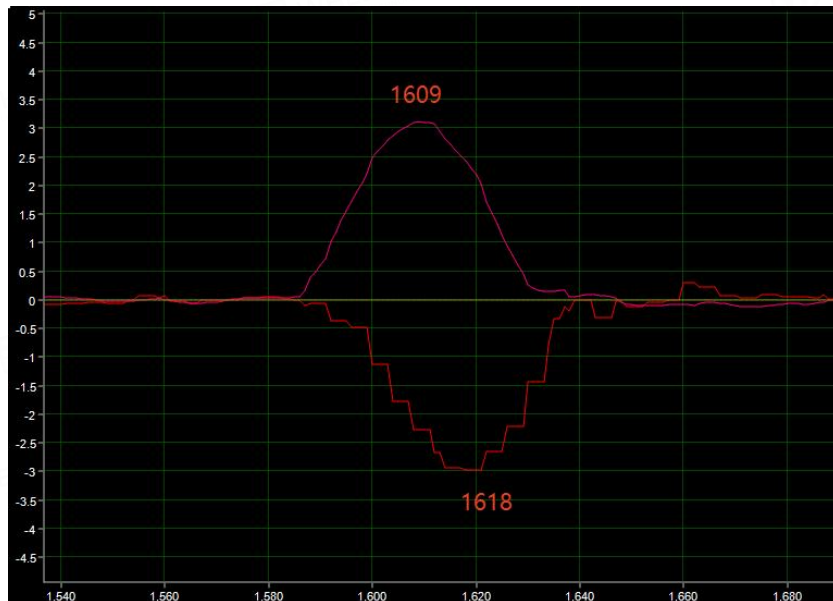


图 4-8 纯旋转状态下的数据采集

可见陀螺仪数据的峰值超前于光流数据的峰值大约 9 个采样周期，因此可以通过一阶低通滤波器让陀螺仪数据延迟大约 9 个采样周期同时降低其中的高频噪声。

● 尺度恢复

假设在旋转补偿的基础上进行分析，当传感器透镜平面离地面高度为 h 的时候，无人机水平向左沿 X 轴正方向移动一段距离，想测得无人机的真实位移距离即 Δl ，光流输出的数据为某一特征点的像素位移量 n_x ，根据三角形相似关系可以得到 Δl 和 n_x 的关系：

$$\frac{\Delta l}{h} = \frac{n_x}{f} \Rightarrow \Delta l = h \cdot \frac{n_x}{f}$$

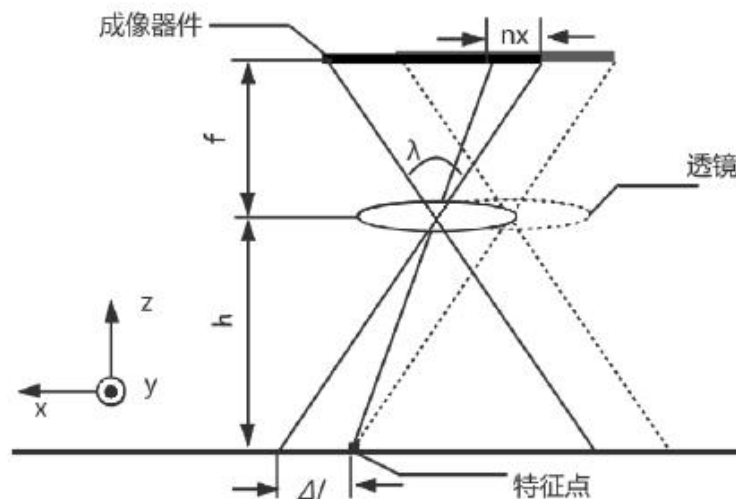


图 4-9 尺度恢复原理图

5.9 传统互补滤波 (CF)

（传统）互补滤波器是一种简洁而高效的信号融合技术，核心思想在于结合两种（或多种）具有互补频率特性传感器的数据，以生成一个更准确、更稳定的估计值。以两种具有互补频率特性的信号为例，一种信号带有高频噪声，另一种信号带有低频噪声，所以我把它们的噪声分别滤除，然后合并，就可以得到了没有噪声的原始信号。

在我们的问题中，需要估计的是无人机的速度，而获得速度的方式有两种：一种是利用光流传感器的数据（在 4.8 节中有介绍），另一种是对加速度（由 IMU 提供）积分得到。光流传感器的数据含有高频噪声，而积分计算容易引入低频噪声，这就很适合利用互补滤波器融合这两种信号获得更可靠的速度信息。下面推

导一下互补滤波器的形式：

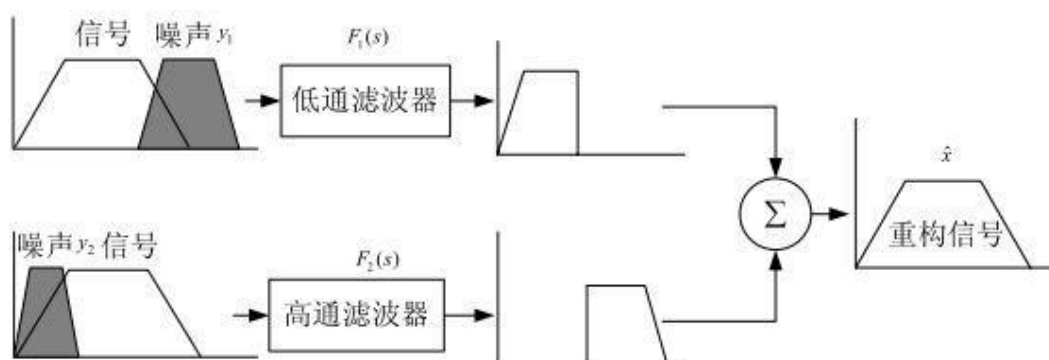


图 4-10 传统互补滤波器

在频域上，

一阶低通滤波器的表达式为：

$$F_1(s) = \frac{1}{\tau \cdot s + 1}$$

一阶高通滤波器的表达式为：

$$F_2(s) = \frac{\tau \cdot s}{\tau \cdot s + 1}$$

则

$$\hat{x}(s) = F_1(s) \cdot \dot{x}_{OF}(s) + F_2(s) \cdot \left(\frac{a(s)}{s} \right) = \frac{\dot{x}_{OF} + \tau \cdot a(s)}{\tau \cdot s + 1}$$

两边同时乘以 $(\tau \cdot s + 1)$ 有：

$$(\tau \cdot s + 1)\hat{x}(s) = \dot{x}_{OF}(s) + \tau \cdot a(s)$$

两边同时进行拉普拉斯逆变换可得：

$$\tau \cdot \hat{\dot{x}}(t) + \hat{x}(t) = \dot{x}_{OF}(t) + \tau \cdot a(t)$$

用前向差分近似 $\hat{\dot{x}}(t)$ 有：

$$\tau \cdot \frac{\hat{x}(t) - \hat{x}(t-1)}{\Delta t} + \hat{x}(t) = \dot{x}_{OF}(t) + \tau \cdot a(t)$$

化简整理得：

$$\hat{x}(t) = \frac{\Delta t}{\tau + \Delta t} \dot{x}_{OF}(t) + \frac{\tau}{\tau + \Delta t} (\hat{x}(t-1) + a(t)\Delta t)$$

令

$$k = \frac{\tau}{\tau + \Delta t}$$

则

$$\hat{x}(t) = (1 - k)x_{OF}(t) + k(\hat{x}(t - 1) + a(t)\Delta t)$$

其中， $x_{OF}(t)$ 为光流传感器的输出值， $a(t)$ 为该方向的加速度值， k 为滤波系数， $\hat{x}(t)$ 为融合后的速度估计值。

注：加速度需要去除重力分量并且需要是在世界坐标系下的表示。

下面利用上位机采集好的数据在 MATLAB 平台上进行试算：

```
clc
clear
close all
%% 读取数据
load('data.mat');
ax = data.ax; % X方向的加速度
vx = data.vx; % X方向的速度
gy = data.gy; % 绕Y轴转动的角速度
h = data.h; % 高度
Ts = 0.005; % 采样间隔
t = 0:Ts:(size(h,2)-1)*Ts; % 时间
%% 一阶低通滤波（角速度）
% 设置滤波系数
alpha = 0.1;
%
% 初始化输出向量
gy_filtered = zeros(size(gy));
gy_filtered(1) = gy(1); % 初始化第一个输出值，避免初始条件问题
%
% 应用递推公式进行滤波
for n = 2:length(gy)
    gy_filtered(n) = alpha * gy(n) + (1 - alpha) * gy_filtered(n-1);
end
%% 旋转补偿
vx_rot_comp = zeros(1, length(vx));
vx_rot_comp(1) = (vx(1) + gy_filtered(1)) * h(1);
for n = 2:length(vx)
    if (vx(n) == vx(n-1))
        vx_rot_comp(n) = vx_rot_comp(n-1);
    else
        vx_rot_comp(n) = (vx(n) + gy_filtered(n)) * h(n);
    end
end
%% 对加速度积分得到速度
```

```

ax2vx = zeros(size(vx));
ax2vx(1) = vx(1); % 初始化第一个输出值，避免初始条件问题
%
% 积分求和
for n = 2:length(ax2vx)
    ax2vx(n) = ax2vx(n-1) + ax(n-1)*Ts;
end
%% 互补滤波
% 互补滤波参数设置
comp_alpha = 0.9; % 互补滤波系数，决定信任加速度计数据的程度
%
% 初始化互补滤波输出
vx_comp = zeros(size(vx));
vx_comp(1) = vx(1); % 初始值
%
% 应用互补滤波
for n = 2:length(vx_comp)
    % 互补滤波公式：融合积分得到的速度和旋转补偿后的速度
    vx_comp(n) = comp_alpha * (vx_comp(n-1) + ax(n)*Ts) + (1 - comp_alpha) *
        vx_rot_comp(n);
end
%% 计算位移
% 通过积分速度得到位移
disp_vx_rot_comp = zeros(size(vx_rot_comp));
disp_ax2vx = zeros(size(ax2vx));
disp_vx_comp = zeros(size(vx_comp));
%
% 使用梯形积分法计算位移
for n = 2:length(vx_rot_comp)
    % 旋转补偿速度的位移
    disp_vx_rot_comp(n) = disp_vx_rot_comp(n-1) + (vx_rot_comp(n) +
        vx_rot_comp(n-1)) * Ts / 2;
    %
    % 加速度积分的位移
    disp_ax2vx(n) = disp_ax2vx(n-1) + (ax2vx(n) + ax2vx(n-1)) * Ts / 2;
    %
    % 互补滤波速度的位移
    disp_vx_comp(n) = disp_vx_comp(n-1) + (vx_comp(n) + vx_comp(n-1)) * Ts / 2;
end
%% 绘制图像
figure('Units', 'normalized', 'Position', [0.25, 0.25, 0.50, 0.50]);
%
% 图 1: 速度比较图
subplot(3, 1, 1);

```

```

plot(t, vx_rot_comp, 'r', 'LineWidth', 1.5);
hold on;
plot(t, ax2vx, 'b', 'LineWidth', 1.5);
plot(t, vx_comp, 'g', 'LineWidth', 1.5);
grid on;
xlim([0 max(t)]);
xlabel('time (s)');
ylabel('velocity (m/s)');
title('Velocity Comparison');
legend('Rotational Compensated', 'Acceleration Integrated', ...
'Complementary Filtered', 'Location', 'southwest');
%
% 图 2: 位移比较图
subplot(3,1,2);
plot(t, disp_vx_rot_comp, 'r', 'LineWidth', 1.5);
hold on;
plot(t, disp_ax2vx, 'b', 'LineWidth', 1.5);
plot(t, disp_vx_comp, 'g', 'LineWidth', 1.5);
grid on;
xlim([0 max(t)]);
xlabel('time (s)');
ylabel('displacement (m)');
title('Displacement Comparison');
legend('Rotational Compensated', 'Acceleration Integrated', ...
'Complementary Filtered', 'Location', 'southwest');
%

```

```

% 图 3: 互补滤波结果 (速度和位移)
subplot(3,1,3);
yyaxis left;
plot(t, vx_comp, 'g', 'LineWidth', 1.5);
ylabel('velocity (m/s)');
yyaxis right;
plot(t, disp_vx_comp, 'm', 'LineWidth', 1.5);
ylabel('displacement (m)');
grid on;
xlim([0 max(t)]);
xlabel('time (s)');
title('Complementary Filter Results');
legend('Velocity', 'Displacement', 'Location', 'southeast');

```

运行后的结果如下所示:

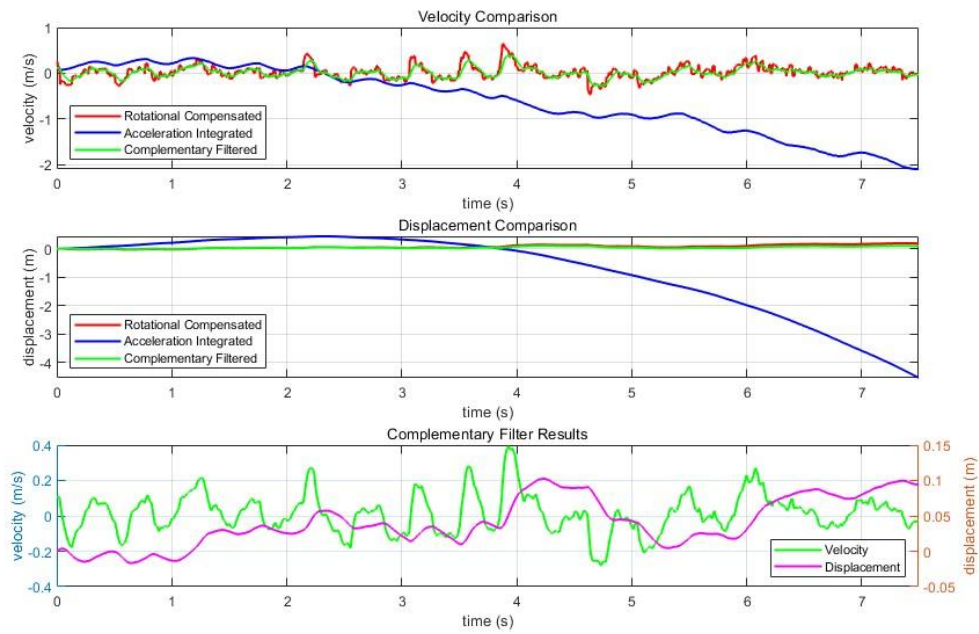


图 4-11 互补滤波后的结果对比